

AI Job Tracker Codebase Explanation

1. What This App Does

This is an AI Job Tracker built with Next.js, React, Supabase, and an OpenAI API route.

A user can:

- Create an account or sign in.
- Add job applications with company, title, status, and notes.
- See dashboard numbers like total applications, interviews, and response rate.
- Edit or delete saved jobs.
- Generate a tailored cover letter from a company, job title, and job description.

The app is mostly one page: `app/page.tsx`.

2. Important Files

- `app/page.tsx`: main homepage. Loads the signed-in user, loads their jobs, calculates dashboard metrics, and renders the UI.
- `components/auth-header.tsx`: sign in, sign up, and sign out UI.
- `components/add-job-form.tsx`: form for adding a new job.
- `components/recent-applications-list.tsx`: shows jobs, edit dialog, and delete button.
- `components/cover-letter-assistant.tsx`: client UI for generating cover letters.
- `app/actions/auth.ts`: server actions for sign in, sign up, and sign out.
- `app/actions/jobs.ts`: server actions for create, update, and delete job records.
- `app/api/cover-letter/route.ts`: backend API route that calls OpenAI.
- `lib/supabase/server.ts`: creates the Supabase server client using cookies.
- `lib/supabase/middleware.ts` and `middleware.ts`: keep Supabase auth sessions fresh.
- `lib/jobs.ts`: job row type and dashboard metric helpers.
- `lib/job-status.ts`: allowed job statuses.
- `supabase/migrations/20260510180000_jobs_user_rls.sql`: database security rules so users only access their own jobs.

3. Main User Flow

1. User visits `/`.
2. `app/page.tsx` checks whether Supabase env vars exist.
3. If signed out, the user sees auth fields from `AuthHeader`.
4. User signs up or signs in using server actions in `app/actions/auth.ts`.
5. Supabase stores the auth session in cookies.
6. The homepage reloads and fetches jobs from the jobs table for that user.
7. User can add, edit, or delete applications.
8. User can open the cover-letter assistant, enter job details, and call `/api/cover-letter`.

9. The API route sends the prompt to OpenAI and returns generated text.

4. How Frontend, Backend, Database, And APIs Connect

The frontend is React components in components/ and the page in app/page.tsx.

The backend is built into Next.js:

- Server actions: app/actions/auth.ts, app/actions/jobs.ts
- API route: app/api/cover-letter/route.ts
- Middleware: middleware.ts

The database is Supabase Postgres. The app uses the jobs table with columns like id, user_id, company, title, status, notes, and created_at.

API connections:

- App to Supabase: via @supabase/ssr.
- App to OpenAI: via fetch("https://api.openai.com/v1/chat/completions") inside app/api/cover-letter/route.ts.

Important uncertainty: code can be inspected locally, but live Supabase policies, tables, and OpenAI keys require your external accounts.

5. Key Code, In Simple Terms

app/page.tsx

- loadJobsForUser(userId): creates a Supabase client, queries the jobs table, filters by user_id, and sorts newest first.
- HomePage(): checks env setup, checks the logged-in user, loads jobs, calculates metrics, and renders the dashboard.
- jobDashboardMetrics(jobs): returns total tracked jobs, number of interview statuses, and response rate.

app/actions/jobs.ts

- requireUserSupabase(): shared guard. It makes sure Supabase is configured and the user is signed in.
- createJob(formData): reads form fields, validates company/title, inserts a row into jobs.
- updateJob(formData): reads form fields, validates them, updates only the matching id and user_id.
- deleteJob(formData): deletes only the job matching both id and current user.
- revalidatePath("/"): tells Next.js to refresh cached page data.

components/add-job-form.tsx

- "use client" means it runs in the browser.
- useActionState(...) connects the form to the server action.
- useRef(...) stores a reference to the form so it can reset after success.
- router.refresh() reloads server-rendered data after adding a job.

components/recent-applications-list.tsx

- `useState<JobRow | null>(null)` tracks which job is being edited.
- `openEdit(job)` saves the selected job and opens a dialog.
- `EditJobForm` submits updates to `updateJob`.
- Delete form calls `deleteJob` after a browser confirmation.

components/cover-letter-assistant.tsx

- Uses `useState` for company, title, job description, loading, error, and generated letter.
- `generate()` sends a POST request to `/api/cover-letter`.
- The backend validates input, calls OpenAI, and returns `{ letter }`.
- `copyLetter()` uses the browser clipboard API.

6. Terms You Should Know

- Props: data passed into a component. Example: `AuthHeader` receives `loggedInEmail`.
- State: data a component remembers while the user interacts. Example: `letter`, `loading`, `error`.
- Hooks: React functions like `useState`, `useEffect`, `useRef`, `useActionState`.
- Routes: URL-based files. `app/page.tsx` is `/`; `app/api/cover-letter/route.ts` is `/api/cover-letter`.
- API calls: network requests. The cover-letter component calls your own API, and your API calls OpenAI.
- Environment variables: secret/config values like `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `OPENAI_API_KEY`.
- Authentication: proving who the user is. Supabase handles sign up, sign in, sign out, cookies, and sessions.
- Database tables: structured storage. This app mainly uses `jobs`.
- Server components: React components that run on the server. `app/page.tsx` is one.
- Client components: React components that run in the browser. Files with "use client".
- Server actions: functions marked "use server" that forms can call securely.
- Middleware: code that runs before page requests; here it refreshes Supabase sessions.
- RLS: Row Level Security. Supabase rules that prevent users from reading/writing other users' rows.

7. Possible Issues Or Cleanup Areas

- `app/layout.tsx` may still have default metadata.
- `README.md` may still be generic and should explain this project.
- Form styles and fields may be repeated between add and edit forms.
- Stronger database TypeScript types would make debugging easier.
- Add tests for auth, job CRUD, and cover-letter API.
- Consider protecting `/api/cover-letter` with authentication too.
- Document the OpenAI model and required env vars.

8. What To Say In Interviews

"I built a full-stack job tracking app with Next.js App Router, React client components, Supabase authentication, Supabase Postgres, row-level security, server actions for CRUD, and an API route that integrates with OpenAI to generate cover letters. Users can securely manage only their own job applications, and the dashboard calculates simple metrics like total applications, interviews, and response rate."

9. Likely Interview Questions With Simple Answers

1. What problem does it solve? It helps users track job applications and generate tailored cover letters.
2. What stack did you use? Next.js, React, TypeScript, Tailwind CSS, Supabase, and OpenAI.
3. How is authentication handled? Supabase Auth handles sign up, sign in, sign out, and sessions through cookies.
4. How do you protect user data? Each job has a `user_id`, server actions filter by `user_id`, and Supabase RLS enforces ownership.
5. What is a server action? A server-side function that a form can call without creating a separate API endpoint.
6. Why use middleware? To keep Supabase auth cookies/session fresh during server-rendered requests.
7. How does the AI feature work? The browser calls `/api/cover-letter`, and that route sends a prompt to OpenAI.
8. What is RLS? Database-level security that controls which rows each user can access.
9. What would you improve next? Add tests, improve typing, add search/filtering, update README/metadata, and protect the AI route with auth.
10. What was the hardest part? Connecting auth, database access, server-rendered data, and client refreshes cleanly.

10. Study Checklist

Be ready to explain:

- What each main file does.
- The sign-in/sign-up flow.
- How a job gets inserted into Supabase.
- Why `user_id` matters.
- What RLS protects.
- Difference between server and client components.
- Difference between server actions and API routes.
- How `router.refresh()` updates the page.
- What env vars are required.
- How the OpenAI cover-letter request works.
- What you would improve if you had more time.